

GNCAirstrike Project Report

GNCAirstrike — Project Report

What this project is

GNCAirstrike is a MATLAB/Simulink simulation of a **guided missile flying from a launch point to a fixed ground target**, modeling the three classic subsystems of a guided weapon:

- **Guidance** — decides *where the missile should point* (what flight-path angle to fly) based on the geometry between the missile and the target.
- **Navigation** — estimates the missile's own state (pitch rate, pitch angle) from noisy sensors using a Kalman filter, since the controller can't use the "true" simulated state directly.
- **Control** — an LQR autopilot that drives the elevator to track the guidance command, using the Kalman filter's state estimates.

On top of this, the missile obeys a **terrain-avoidance safety rule**: if it gets within 2 km (horizontal) of a known obstacle, it forces a climb regardless of what guidance is asking for, then resumes normal guidance once clear.

The simulation runs in the vertical (longitudinal) plane only — pitch and altitude, no turning/lateral motion — over real launch/target coordinates (latitude, longitude, elevation), using great-circle (haversine) distance for the horizontal geometry.

Guidance law: Proportional Navigation (PN)

The core of this session's work was replacing the original guidance law — which simply pointed the missile along the straight-line bearing to the target's *current* position — with **true Proportional Navigation**, the guidance law real missiles actually use:

$$a_{\text{cmd}} = N' \cdot V_c \cdot \lambda_{\text{dot}}$$

- λ — the line-of-sight (LOS) angle from missile to target
- λ_{dot} — how fast that LOS angle is rotating
- V_c — closing velocity (how fast the missile-target range is shrinking)
- N' — the navigation constant (a tunable gain, set to 3.5 here)

The intuition: if the LOS angle is rotating, the missile isn't on a collision course, so PN commands an acceleration proportional to that rotation rate to drive λ_{dot} toward zero — the classic condition for a collision-course intercept. This command is integrated into a flight-path-angle reference that the existing outer control loop already knows how to track, so the inner LQR/Kalman loop needed no redesign.

The terrain override sits on top of this: while within 2 km of the obstacle, PN is suspended (its internal state is held, not advanced) and a fixed 5° climb is commanded instead; PN resumes cleanly from where it left off once past the obstacle.

What had to be fixed to make it work

Two pre-existing bugs in the base model were discovered and fixed (unrelated to the guidance law itself, but blocking *any* guidance law from converging):

1. **Altitude integrator sign** — the model uses a “Down-positive” NED convention internally, but the vertical-velocity path into the altitude integrator was missing the corresponding sign flip. Fixed with a single gain block.
2. **Outer-loop gain magnitude** — the outer proportional control loop feeds directly into the elevator command as a bias, and the plant is very sensitive to that bias; the original gain overdrove the elevator and diverged. Retuned (magnitude only, sign unchanged) using the closed-loop DC gain and a 5-second target time constant.

The LQR gains and Kalman filter design were **not** touched or redesigned — both were regression-verified to be identical (eigenvalues, gains) before and after all changes.

Validation: Monte Carlo simulation

To check that the guidance law works across a range of conditions — not just the one nominal case — `monte_carlo.m` runs the closed-loop simulation **300 times**, each time randomizing:

- Launch position (latitude/longitude, small scatter)
- Target position (latitude/longitude, small scatter)
- Process and measurement noise seeds (the Kalman filter’s sensor/process noise)

For each trial it records: closest approach to the target (“miss distance”), whether the terrain override fired, whether the elevator saturated, final altitude, flight time, and Kalman filter tracking error.

Results (N = 300)

Metric	Value
Success rate (miss $\leq$ 1000 m)	100%
Mean miss distance	581.1 m
Median miss distance (CEP50)	554.8 m
Worst-case miss distance	957.7 m
Terrain override fired	97.3% of trials
Control saturation	0% of trials

Every one of the 300 randomized trials hit the target under the 1000 m threshold, the terrain override reliably triggers and guidance re-converges cleanly after it, and the elevator never saturates — meaning the retuned control gain has margin across the tested envelope.

What this project does *not* do (explicitly out of scope)

- No obstacle-avoidance path planning — the terrain rule is a simple reactive override, not a planned route around obstacles.
- No lateral-directional or full 6-DOF motion — this is a longitudinal (vertical-plane) simulation only.
- No redesign of the LQR or Kalman filter — both are the original design, only the guidance law feeding them changed.

File guide

File	Purpose
<code>model.m</code>	Defines mission geometry, LQR/Kalman gains, and mission constants (run this before simulating)
<code>simulink_model.slx</code>	The closed-loop guidance/navigation/control simulation
<code>simulink_model_FG.slx</code> , <code>runfg.bat</code>	FlightGear visualization variant

	(pre-existing, not modified)
monte_carlo.m	Runs the 300-trial randomized validation batch
monte_carlo_results.csv / .mat	Per-trial raw results from the Monte Carlo run
miss_distance_histogram.png, trajectory_overlay.png	Monte Carlo result plots
RESULTS.md	Monte Carlo results write-up
PROGRESS.md	Detailed technical log of how the guidance law and bug fixes were derived
SUMMARY.md / SUMMARY.pdf	Earlier summary of the specific code/model changes made
